



Chapter 13: Overloading- PART II

Objectives

- In this chapter, you will
 - Learn about `friend` functions
 - Learn how to overload operators as `non-friend` functions
 - Learn about overloading rules
 - Become familiar with the requirements for classes with pointer member variables

Overloading Unary and Binary Operators

- If unary and binary operators are overloaded using member functions:
 - Unary operators have no parameters
 - Binary operators have only one parameter

Operator Overloading as Friend Functions

Friend Functions of Classes

- Friend function (of a class): a nonmember function of the class that has access to all the members of the class
- Use the reserved word `friend` in the function prototype in the class definition

```
class classIllusFriend
{
    friend void two(/*parameters*/);
    .
    .
    .
};
```

Definition of a friend Function

- When writing the friend function definition
 - The name of the class and the scope resolution operator are not used

```
void two(/*parameters*/)
{
    .
    .
    .
}
```

- "friend" doesn't appear in function definition

Operator Functions as Member and Nonmember Functions

- If `op` is overloaded for `opOverClass`:
 - If the leftmost operand of `op` is an object of a different type, the overloading function must be a nonmember (friend) of the class
 - If the overloading function for `op` is a member of `opOverClass`, then when applying `op` on objects of type `opOverClass`, the leftmost operand must be of type `opOverClass`

Overloading as Friend Functions: Addition with Constant Value

- **Example: 10+amount1**

```
//other declarations
//Inside the class, we should the following line of code (prototype)

friend currency operator+(long d, currency r);

//other declarations
```

```
//outside the class
currency operator+(long d, currency r)
{
    currency temp (d+r.dollars, r.cents);
    return temp;
}

int main()
{
    currency total, amount1(17,50);
    total = 10+amount1;
    total.print();
}
```

OUTPUT
\$27.50

Overloading as Friend Functions: Insertion Operator (<<)

- Overloading insertion operator (<<)

```
//other declarations
//Inside the class, we should the following line of code (prototype)

friend ostream& operator<<(ostream& o, currency cur);

//other declarations
```

```
//outside the class
ostream& operator<<(ostream& o, currency r)
{
    o<<'<math>\$</math>'<<r.dollars<<'.'<<r.cents<<endl;
    return o;
}

int main()
{
    currency amount(100,50);
    cout<<amount;
}
```

OUTPUT
\$100.50

Overloading as Friend Functions: Extraction Operator >>

- Overloading extraction operator (>>)

```
//other declarations
//Inside the class, we should the following line of code (prototype)

friend istream& operator>>(istream& i, currency& cur);

//other declarations
```

```
//outside the class
istream& operator>>(istream& i, currency& r)
{
    long d, c;
    cout<<"Enter dollars: "<<endl;
    i>>d;
    cout<<"Enter cents: "<<endl;
    i>>c;
    r.setAmount(d,c);
    return i;
}

int main()
{
    currency amount;
    cin>>amount;
    amount.print();
}
```

OUTPUT

```
Enter dollars:
10
Enter cents:
20
$10.20
```

Overloading Binary Operators

- Member binary operator (e.g., `+` or `==`) can be overloaded as a `friend` function with two parameters
- Member unary operator can be overloaded as a `friend` function with one parameter.

Overloading the binary Arithmetic or Relational Operators as Nonmember Functions

- Function prototype (included in class definition):

```
friend returnType operator#(const className&,  
                             const className&);
```

- Function definition:

```
returnType operator#(const className& firstObject,  
                     const className& secondObject)  
{  
    //algorithm to perform the operation  
  
    return value;  
}
```

Operator Overloading as Non-Friend and Non-Member Functions

Overloading Operators as Non-Friend and Non-Member Function

- We can overload an operator with a function that is not member and not a friend of the class
- We can use this way for overloading when there is no need to access private members

Overloading Operators as Non-Friend and Non-Member Function [cont]

```
//outside the class/non-member and non-friend
currency operator+(long dol, currency r)
{
    long d,c;
    r.getAmount(d,c);

    currency temp(d+dol,c);
    return temp;
}

int main()
{
    currency total, amount1(17,50);
    total = 10+amount1;
    total.print();
}
```

OUTPUT

\$27.50

Classes and Pointer Member Variables (Revisited)

- Recall that assignment operator copies member variables from one object to another of the same type
 - Does not work well with pointer member variables
- Classes with pointer member variables must:
 - Explicitly overload the assignment operator
 - Include the copy constructor
 - Include the destructor

Overloading Rules

- We cannot create new operators
- We cannot change the basic meaning of an operator (we cannot redefine the plus(+) operator to subtract one value from other)
- The overloaded operator must have at least one operand that is of user-defined type
- We cannot overload: `.` `.*` `::` `?:` `sizeof`
- We cannot use friend functions to overload certain operators however member functions can be used to overload them: `=` `()` `[]` `->`
- Cannot change precedence or associativity
- Default parameters cannot be used
- Cannot change number of parameters

THE END